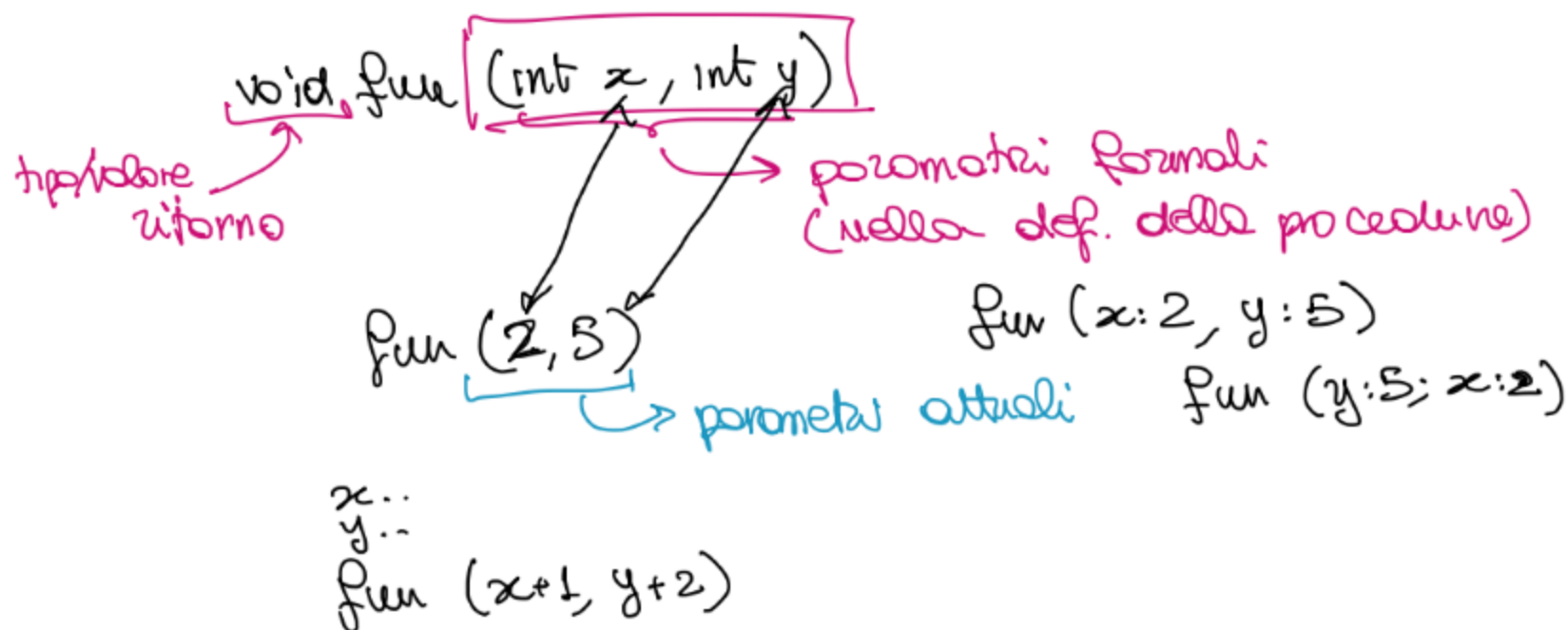


Passaggio dei parametri

↳ parametri come "input" della procedura che permettono una comunicazione tra chiamante e chiamato



`fun (2, w)`

Aspetti progettuali del passaggio di parametri

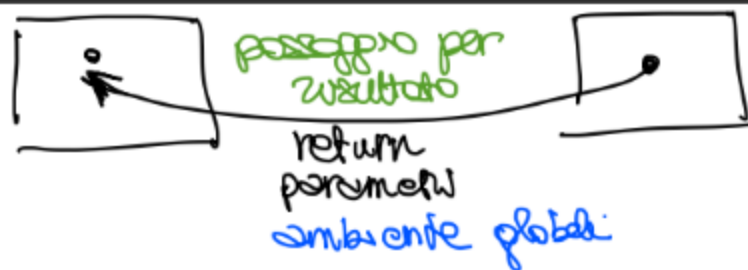
- ① Modalità (valore, riferimento, risultato, valore-risultato)
- ② Associazione parametri attuali e formali

Modalità di passaggio di parametri : direzioni

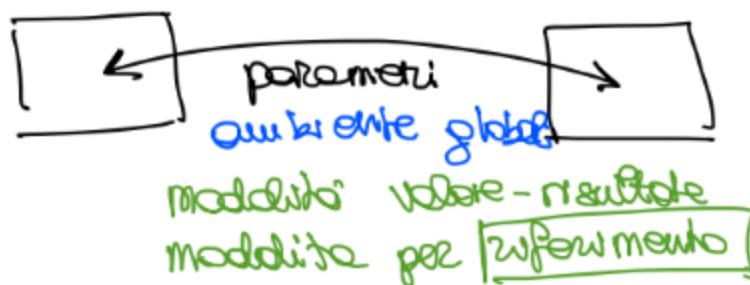
In-mode : chiamante → chiamato



Out-mode : chiamato → chiamante



Inout-mode: chiamante $\longleftrightarrow$ chiamato



Passaggio di parametri per valore

Il valore del parametro attuale (eventualmente ottenuto come valutazione di una espressione)

viene passato come valore iniziale del corrispondente parametro formale

→ implementato creando una copia del valore dell'attuale nel formale

fun ($x+1$, $y+2$)

→ il parametro attuale è un r-value (quello che possiamo scrivere o di cui un assegnamento)

permette una certa variabilità nella scrittura dei parametri attuali

svantaggio $\rightarrow$ utilizzo della memoria

```
void foo(int x) { x = x + 1; }
...
y = 1
foo(y + 1)
```

→ chiamata a foo alloca x nell'ambiente locale di foo

→ alla chiamata $y+1$ viene valutato (2) e il valore viene copiato

nella cella riferita da x

⇒ non c'è nessun legame tra x e y

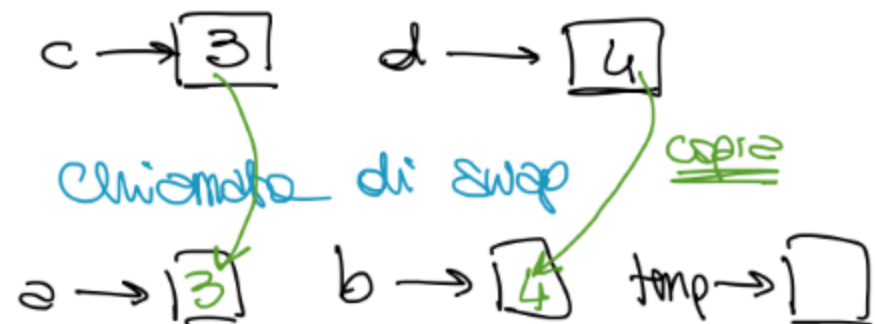
→ alla terminazione di foo il RDA contenente x viene distrutto perdendo tutte le modifiche fatte a x

⇒ questa metodologia non permette di comunicare al chiamante attraverso i parametri

main

```
void swap (value int a, value int b){  
    int tmp = a;  
    a = b;  
    b = tmp;  
}  
  
:  
int c = 3;  
int d = 4;  
swap (c, d);
```

Esecuzione main



Esecuzione di swap

a → [4] b → [3] tmp → [3]

Terminazione di swap

Memoria:

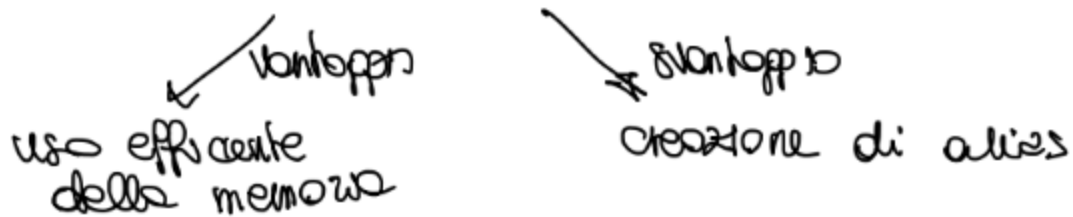
c → [3]	d → [4]	↑ Rimane inalterata
a → [4]	b → [3]	

Eliminazione RDA per swap

Passaggio per riferimento

↳ Viene passato un cammino di accesso al parametro funzione

In-out



⇒ I parametri attuali possono essere solo l-value (riferimenti)

```
void foo(reference int x) { x = x + 1; }
```

```
⋮  
y = 2;  
foo(y);
```

→ Alla chiamata si crea RfA con cella per x

→ Alla chiamata x diventa un riferimento alla cella riferita da y



→ Alla terminazione viene distrutto RfA contenente il riferimento x

⇒ Tutte le informazioni fatte a memoria vengono riferibili da y

```
void swap(ref int a, ref int b) {
```

```
    int tmp = a;
```

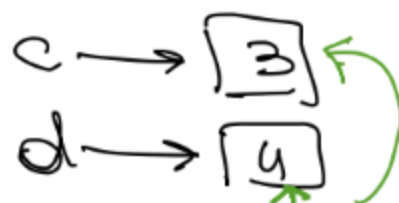
```
    a = b;
```

```
    b = tmp;
```

```
}
```

```
int c = 3;  
int d = 4;
```

Esecuzione main



`swap(c,d);`

Chiamata di swap

`tmp` → `a` `b`

Esecuzione di swap

`tmp` → 3 `c` → 4 ← `a`
`d` → 3 ← `b`

Terminazione di swap

~~`tmp` → 3~~ ~~`c` → 4 ← `a`~~
~~`d` → 3 ← `b`~~

Esercizio 1 → sul passaggio di parametri

Def →
passaggio di parametri per valore
passaggio di parametri per riferimento

```
void fun (int a, int b) {  
    a = a + 5;  
    b = 2b;  
}
```

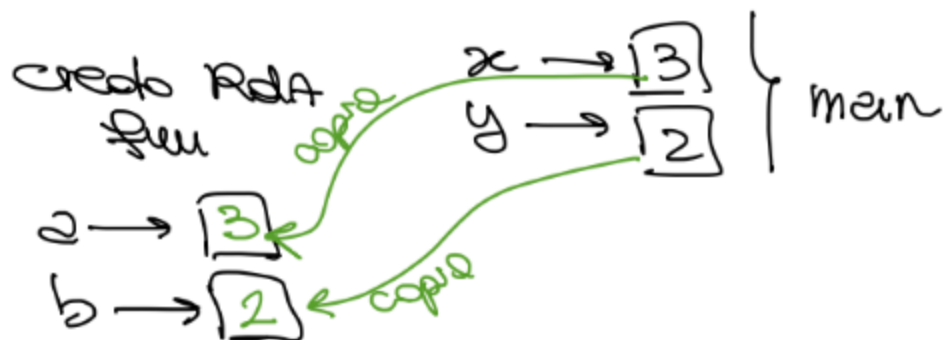
```
void main () {  
    int x=3, y=2;  
    fun (x,y);  
}
```

Chiamata per valore

Esecuzione main

RdA main: $x \rightarrow [3]$
 $y \rightarrow [2]$

Chiamata fun



Esecuzione fun

RdA main: $x \rightarrow [3]$
 $y \rightarrow [2]$

RdA fun: $a \rightarrow [8]$
 $b \rightarrow [4]$

Terminazione fun

RdA main: $x \rightarrow [3]$
 $y \rightarrow [2]$

~~RdA fun: $a \rightarrow [8]$
 $b \rightarrow [4]$~~

Valori di x e y alla terminazione sono
 $x=3$ e $y=2$

Passaggio per riferimento

Esecuzione main: RdA Main: $x \rightarrow [3]$
 $y \rightarrow [2]$

Chiamata fun: RdA main: $x \rightarrow [3]$
 $y \rightarrow [2]$

RdA fun: $a \rightarrow []$
 $b \rightarrow []$

Esecuzione fun: RdA main: $x \rightarrow \boxed{3}$ ⁸
 $y \rightarrow \boxed{2}$ ⁴
 RdA fun: $a \rightarrow \boxed{\cdot}$
 $b \rightarrow \boxed{\cdot}$

Terminazione fun: RdA main: $x \rightarrow \boxed{8}$
 $y \rightarrow \boxed{4}$
~~RdA fun: $a \rightarrow \boxed{\cdot}$
 $b \rightarrow \boxed{\cdot}$~~

Allo terminazione $x=8$ e $y=4$ quindi le modifiche avvenute in fun rimangono nel main

Esercizio 2

```
void pippo (int a, int b) {
    a = 5 + b;
    b = 3 * a;
}
```

```
void main() {
    int x = 3, y = 2;
    pippo(x, x);
}
```

Passaggio per valore

Esecuzione main: RdA main: $x \rightarrow \boxed{3}$
 $y \rightarrow \boxed{2}$

Chiamata pippo: RdA main: $x \rightarrow \boxed{3}$ $y \rightarrow \boxed{2}$
 RdA pippo: $a \rightarrow \boxed{3}$ $b \rightarrow \boxed{3}$

Esecuzione pippo: RdA main: $x \rightarrow [3]$ $y \rightarrow [2]$
RdA pippo: $a \rightarrow [8]$ $b \rightarrow [24]$

Terminazione pippo: RdA main: $x \rightarrow [3]$ $y \rightarrow [2]$
~~RdA pippo: $a \rightarrow [8]$ $b \rightarrow [24]$~~

I valori nel main di x e y rimangono inalterati

Passaggio per riferimento

Esecuzione main: RdA main: $x \rightarrow [3]$ $y \rightarrow [2]$

Chiamata pippo: RdA main: $x \rightarrow [3]$ $y \rightarrow [2]$
RdA pippo: $a \rightarrow []$ $b \rightarrow []$

Esecuzione pippo: RdA main: $x \rightarrow [3]$ $y \rightarrow [2]$
RdA pippo: $a \rightarrow []$ $b \rightarrow []$

Terminazione pippo: RdA main: $x \rightarrow [24]$ $y \rightarrow [2]$
~~RdA pippo: $a \rightarrow []$ $b \rightarrow []$~~

Allo terminazione $x=24$ e $y=2$.

Esercizio 3 (completare)

void fun (int first, int second) {

first += first;

second += second;

}

void main () {

int list[] = {3, 5};

fun (list[0], list[1]);

}